Implement Kafka retry topics and dead-letter queues :

For your Netflix Adaptive Bitrate Streaming project (Spring Boot + Kafka + FFmpeg + S3), implementing Kafka retry topics and a Dead Letter Queue (DLQ) is a good production-oriented improvement.

It helps when a video fails to encode, S3 temporarily becomes unavailable, or a consumer encounters a processing error.

Instead of endlessly retrying the same video and getting stuck, you can retry it a limited number of times and then isolate it for investigation.

```
package com.netflix.encodingservice.kafka;

import org.springframework.kafka.annotation.DltHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.annotation.RetryableTopic;
import org.springframework.retry.annotation.Backoff;
import org.springframework.stereotype.Component;

import com.netflix.encodingservice.event.VideoUploadedEvent;
import com.netflix.encodingservice.service.VideoEncodingService;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

@Component
@RequiredArgsConstructor
@Slf4j
public class VideoEncodingConsumer {

    private final VideoEncodingService encodingService;

    @RetryableTopic(
        attempts = "4",
        backoff = @Backoff(
            delay = 1000,
            multiplier = 2.0,
            maxDelay = 5000
        ),
        retryTopicSuffix = "-retry",
        dltTopicSuffix = "-dlt"
    )
    @KafkaListener(
        topics = "video.uploaded",
```

```
        groupId = "encoding-service-group"
    )
    public void consume(VideoUploadedEvent event) {

        log.info("Received video event: {}", event.movieId());

        // Download the source video from S3,
        // encode using FFmpeg, and upload HLS files.
        encodingService.encode(event);
    }

    @DltHandler
    public void handleDeadLetter(VideoUploadedEvent event) {

        log.error(
            "Video encoding permanently failed. movieId={}, s3Key={}",
            event.movieId(),
            event.s3Key()
        );

        // Persist failure details and mark the video FAILED.
        // Alert/monitoring can be triggered here.
    }
}
```

the Encoding Service can receive the event up to 4 times with `attempts = "4"`.

But the Video Service publishes the original event only once.

- Attempt 1: Original event from `video.uploaded`.
- Attempt 2: Retry after 1 second.
- Attempt 3: Retry after 2 seconds.
- Attempt 4: Retry after 4 seconds.
- If all 4 attempts fail → event goes to the Dead Letter Topic (`video.uploaded-dlt`).

Add durable encoding-job state transitions and automatic recovery.

For your Netflix Adaptive Bitrate Streaming project, durable encoding-job state transitions and automatic recovery means your system remembers exactly where each video is in the encoding pipeline, even if Kafka, your Spring Boot service, or FFmpeg crashes.

1. Why you need this

Imagine your two-minute video is being converted into four HLS qualities:

- 1080p
- 720p
- 480p
- 360p

FFmpeg successfully generates the files, and your Encoding Service uploads some segments to S3. Then the service crashes before publishing `VideoEncodedEvent`.

When the service restarts, Kafka might redeliver the original event. Without durable job state, your service may encode the entire video again, duplicate work, or leave the video stuck in `ENCODING`.

How the complete recovery works

Imagine your two-minute video crashes during 720p encoding.

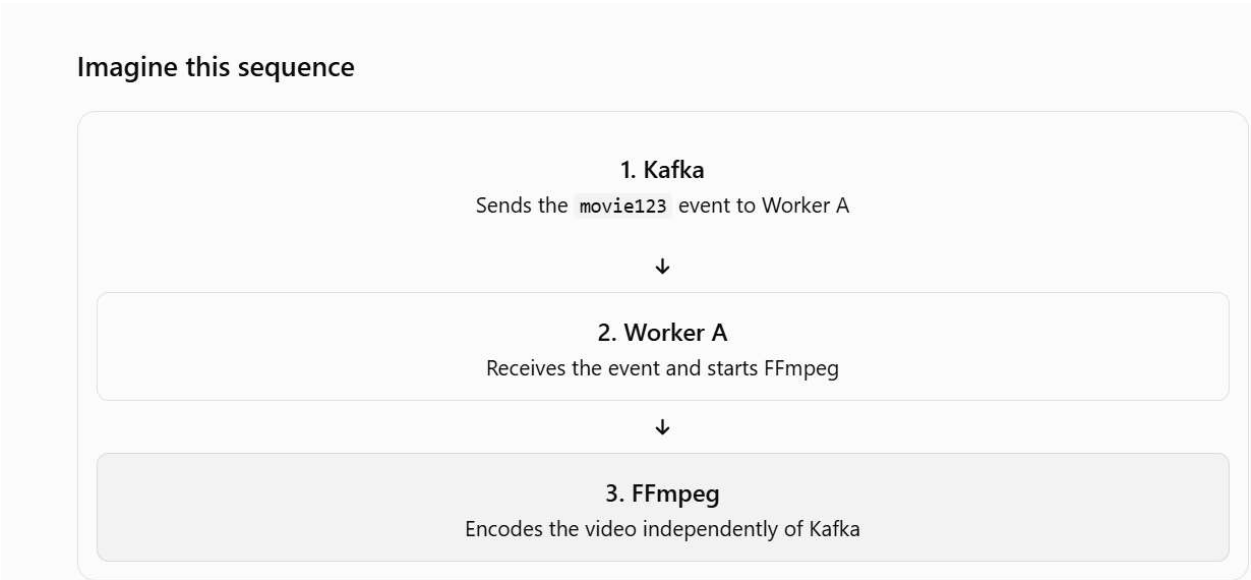
1. PostgreSQL retains the job state even though the service stops.
2. The worker's lease expires.
3. The recovery task detects the abandoned job.
4. A worker safely claims it again.
5. The encoding pipeline resumes or restarts the job, depending on which outputs can be verified.
6. Once all HLS outputs are valid, the job becomes `COMPLETED` and the video becomes `READY`.

Add atomic idempotency controls for concurrent encoding requests :

In your Netflix Adaptive Bitrate Streaming project, atomic idempotency controls for concurrent encoding requests means preventing the same video from being encoded multiple times when duplicate Kafka events arrive or multiple workers attempt to process it simultaneously.

Suppose worker A (threads) is unresponsive with kafka,then kafka will retry the event and again starts encoding,here two times encoding happens so,,to stop that atomic idempotency is required.

Much clean understand: so my main confusion is if kafks consumer or kafka is unresponsvie why it will retry..coz kafka is not related with encoding process ryt



Now imagine Worker A becomes unresponsive to Kafka while FFmpeg is still encoding.

Why might Kafka deliver the event again?

Because Kafka cannot automatically know that FFmpeg is still running or that the video has finished encoding. Kafka tracks the consumer's progress through the event log using offsets.

If Worker A has not committed the event's offset and Kafka reassigns the partition to Worker B, Worker B may receive that event again.

Kafka is not retrying the encoding. It's making the unacknowledged event available for processing again.

The possible reasons for duplication:

If the encoding limit is 2 min or its fine no need of mentioning time limit for encoding, and if poll interval is 5 min, and offset updates to kafka from worker ,then its fine video encoded successfully happy..

1) If no updation of offset within 5 min poll() and encoding still happening,,then kafka will retry ,then here video encoding duplication happens
2) Worker get stuck or unresponsive
3) Worker finishes encoding but crashes before committing the offset.

max.poll.interval.ms = 300000 ,max limit is 5 min lets..can call n number of poll(),if time limit exceeded then another consumer will takes places in the same group.

If event A is received and its task is processing still not complete and time limit exceeded then another consumer in the same group will receives the event A again,and now starts the processing the task again.

Here duplication of tasks processing happens..to avoid that we use idempotency concept

Bookish language explanation:

1. `max.poll.interval.ms = 300000` means the maximum allowed gap between calls to `poll()` is 5 minutes.
2. Consumer A receives Event A and starts processing it using FFmpeg.
3. Processing takes longer than 5 minutes, and Consumer A doesn't call `poll()` again.
4. Kafka may trigger a rebalance and assign the partition to Consumer B.
5. If Event A's offset is uncommitted, Consumer B may receive Event A again and start processing it while Consumer A's FFmpeg process is still running.

Result: Both consumers could process the same task, causing duplicate encoding.

What does idempotency mean?

An operation is idempotent when repeating it produces the same intended result as performing it once.

For your project:

- First request for `movie-101` → create and process its encoding job.
- Duplicate request for `movie-101` → recognize the existing job and avoid creating duplicate work.
- Request after successful completion → keep the job completed rather than encoding it again unnecessarily.

However, idempotency alone is not enough. Two workers could both check that a job is available before either updates its status.

That's why you need atomic controls.

Why a normal status check is not enough

Consider this code:

```
EncodingJob job = repository.findByMovieId(movieId);

if (job.getStatus().equals("PENDING")) {
    job.setStatus("PROCESSING");
    repository.save(job);

    ffmpegService.encode(movieId);
}
```

This looks reasonable, but it has a race condition.

1. Worker A reads `PENDING`.
2. Worker B also reads `PENDING`.
3. Worker A updates the status to `PROCESSING`.
4. Worker B also updates it to `PROCESSING`.
5. Both workers start FFmpeg.

Fix it with an atomic database claim

With PostgreSQL, you can use a conditional `UPDATE` so the status check and state change happen as one atomic database operation.

```
UPDATE encoding_jobs
SET status = 'PROCESSING',
    worker_id = :workerId,
    lease_until = CURRENT_TIMESTAMP + INTERVAL '5 minutes',
    attempts = attempts + 1
WHERE movie_id = :movieId
```

```
  AND status = 'PENDING'
RETURNING movie_id;
```

Introduce integration tests for S3, Kafka, Redis, and PostgreSQL interactions.

integration tests verify that your Spring Boot services work correctly with real or test instances of S3, Kafka, Redis, and PostgreSQL—not just that individual Java methods work in isolation.

What to test ;

1. AWS S3 — Video storage

2. Kafka — Event delivery

3. Redis — Streaming URL cache

4. PostgreSQL — Metadata persistence

Which tools should you use?

Component	Recommended testing approach
Spring Boot services	JUnit 5 + Spring Boot Test
S3	LocalStack or an isolated AWS test bucket
Kafka	Testcontainers with Kafka
Redis	Testcontainers with Redis

PostgreSQL	Testcontainers with PostgreSQL
Build and test execution	Maven

Add automated builds, deployment pipelines, metrics, and distributed tracing.

1. Ci cd
2. Metrics: health checks ,latency ,failed counts, seconds(time taken for task)